Lab_Assignment_8.2

AI Assisted Coding B37 2303A52077

Task 1 – Test-Driven Development for Even/Odd Number Validator

Scenario:

Use AI tools to first generate test cases for a function is_even(n) and then implement the function so that it satisfies all generated tests.

Requirements:

• Input must be an integer

• Handle zero, negative numbers, and large integers

Example Test Scenarios:

is_even(2) → True

is_even(7) → False

is_even(0) → True

is_even(-4) → True

is_even(9) → False

Expected Output -1

• A correctly implemented is_even() function that passes all AI-generated test cases

Code:

```
def is_even(num):
    return num & 1 == 0
```

Explanation:

This function uses a bitwise AND operation with 1 to check if a number is even.

For any integer, the least significant bit (rightmost bit) is 1 if the number is odd and 0 if the number is even. By performing a bitwise AND with 1, we isolate this bit.

If the result is 0, the number is even; if it's 1, the number is odd.

Args: num (int): The number to check for evenness.

Returns: bool: True if the number is even, False if the number is odd.

Prompt: #genrate test cases with integer values as input

Output:

0 is even: True 1 is even: False

2 is even: True 3 is even: False

4 is even: True

5 is even: False

6 is even: True

7 is even: False

8 is even: True

9 is even: False

10 is even: True

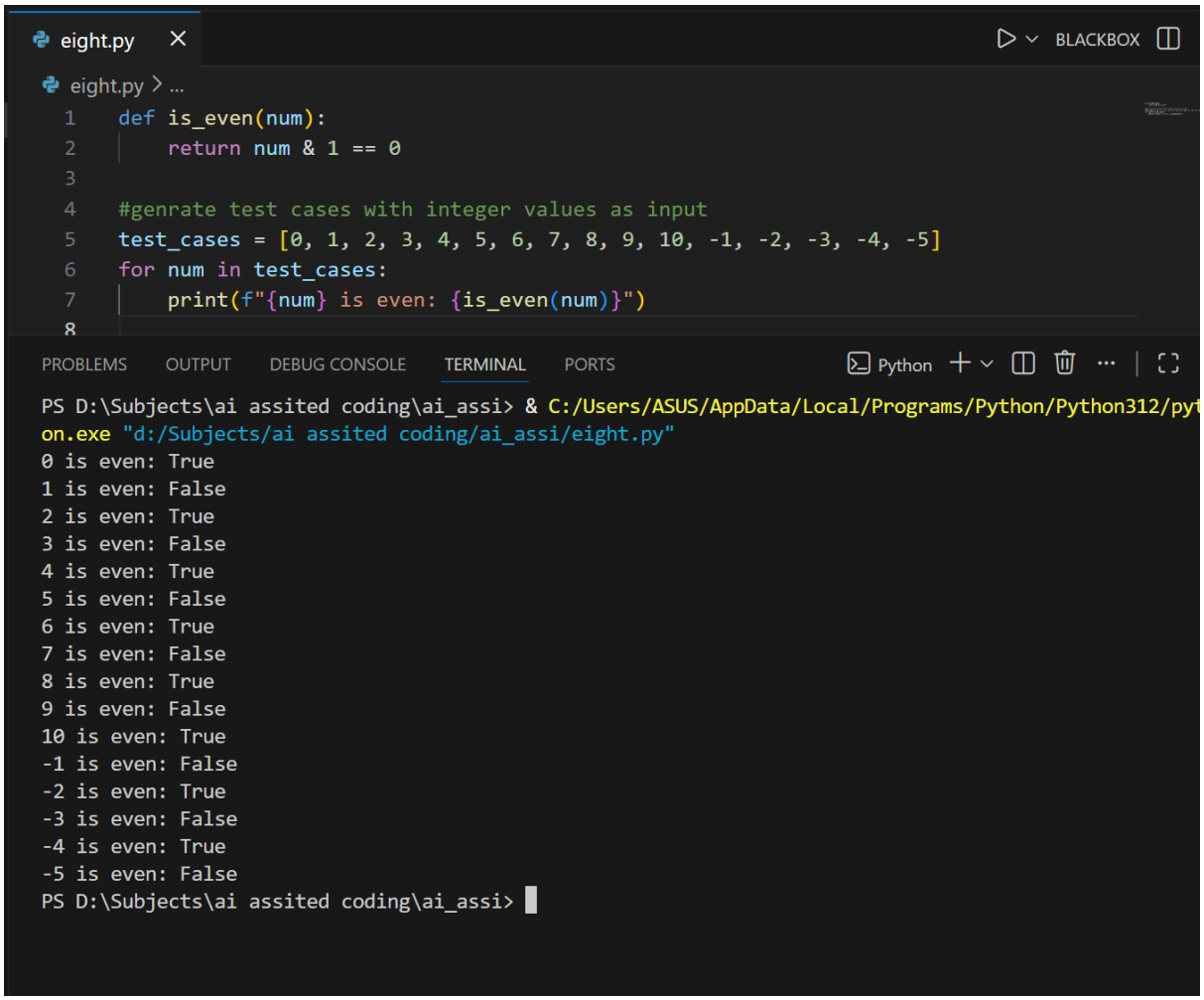
-1 is even: False

-2 is even: True

-3 is even: False

-4 is even: True

-5 is even: False



Task 2 – Test-Driven Development for String Case Converter

Scenario:

• Ask AI to generate test cases for two functions:

• to_uppercase(text)

• to_lowercase(text)

Requirements:

• Handle empty strings

• Handle mixed-case input

• Handle invalid inputs such as numbers or None

Example Test Scenarios:

to_uppercase("ai coding") → "AI CODING"

to_lowercase("TEST") → "test"

to_uppercase("") → ""

to_lowercase(None) → Error or safe handling

Expected Output -2

• Two string conversion functions that pass all AI-generated test

cases with safe input handling.

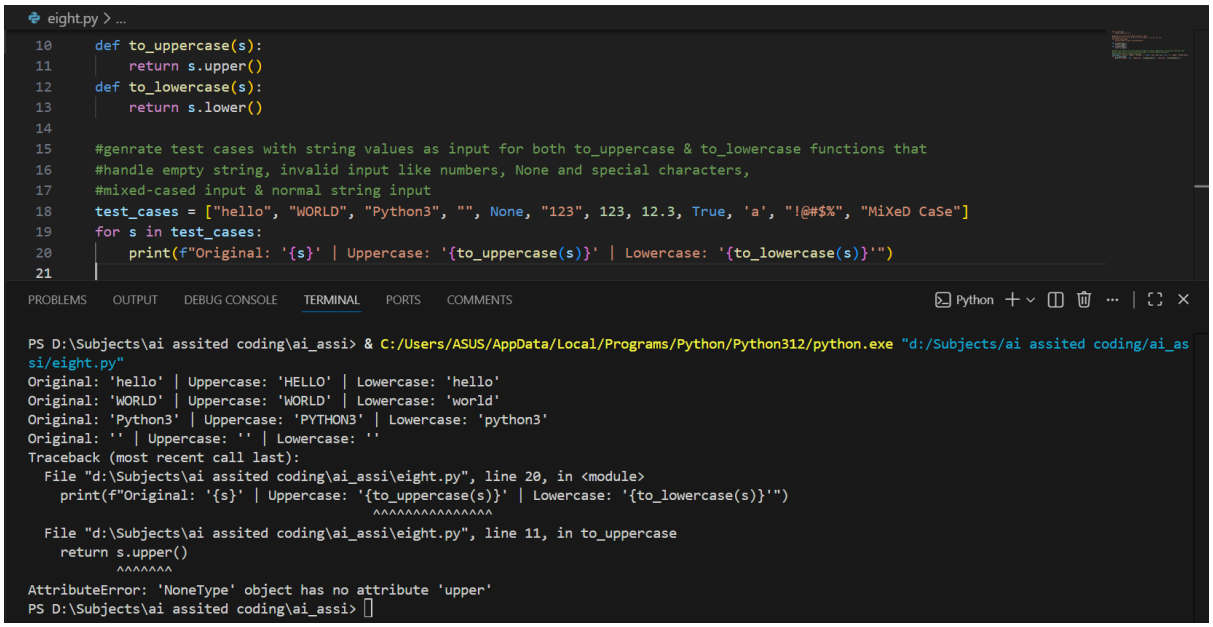
Code:

```
def to_uppercase(s):
    return s.upper()
def to_lowercase(s):
    return s.lower()
```

#Testcase prompt:

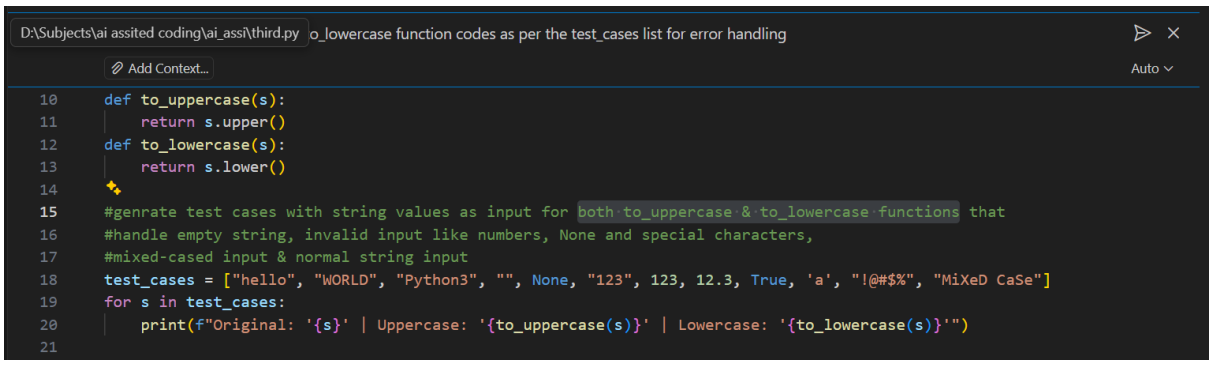
```
#genrate test cases with string values as input for both to_uppercase & to_lowercase functions that
#handle empty string, invalid input like numbers, None and special characters,
#mixed-cased input & normal string input
```

```
test_cases = ["hello", "WORLD", "Python3", "", None, "123", 123, 12.3, True, 'a', "!@#$%", "MiXeD CaSe"]

for s in test_cases:

    print(f"Original: '{s}' | Uppercase: '{to_uppercase(s)}' | Lowercase: '{to_lowercase(s)}'")
```



Function code modification:

Prompt: update both to_uppercase & to_lowercase function codes as per the test_cases list for error handling



Final code & Output:

```python
    def to_uppercase(s):
        if isinstance(s, str):
            return s.upper()
        return None

    def to_lowercase(s):
        if isinstance(s, str):
            return s.lower()
        return None

    #genrate test cases with string values as input for both to_uppercase & to_lowercase functions that
    #handle empty string, invalid input like numbers, None and special characters,
    #mixed-cased input & normal string input
    test_cases = ["hello", "WORLD", "Python3", "", None, "123", 123, 12.3, True, 'a', "!@#$%", "MiXeD CaSe"]
    for s in test_cases:
        print(f"Original: '{s}' | Uppercase: '{to_uppercase(s)}' | Lowercase: '{to_lowercase(s)}'")
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  COMMENTS

si/eight.py"
Original: 'hello' | Uppercase: 'HELLO' | Lowercase: 'hello'
Original: 'WORLD' | Uppercase: 'WORLD' | Lowercase: 'world'
Original: 'Python3' | Uppercase: 'PYTHON3' | Lowercase: 'python3'
Original: '' | Uppercase: '' | Lowercase: ''
Original: 'None' | Uppercase: 'None' | Lowercase: 'None'
Original: '123' | Uppercase: '123' | Lowercase: '123'
Original: '123' | Uppercase: 'None' | Lowercase: 'None'
Original: '12.3' | Uppercase: 'None' | Lowercase: 'None'
Original: 'True' | Uppercase: 'None' | Lowercase: 'None'
Original: 'a' | Uppercase: 'A' | Lowercase: 'a'
Original: '!@#$%' | Uppercase: '!@#$%' | Lowercase: '!@#$%'
Original: 'MiXeD CaSe' | Uppercase: 'MIXED CASE' | Lowercase: 'mixed case'
PS D:\Subjects\ai assited coding\ai_assi>
```

Explanation:

Function: `to_uppercase(s)`

- Checks if input `s` is a string using `isinstance(s, str)`

- If true, converts to uppercase using `.upper()` method

- If false, returns `None`

Function: ‘to_lowercase(s)’

- Checks if input `s` is a string using `isinstance(s, str)`

- If true, converts to lowercase using `.lower()` method

- If false, returns `None`

Test Cases:

The list contains various inputs to test edge cases:

- Normal strings: `"hello"`, `"WORLD"`, `"Python3"`

- Empty string: `""`

- Invalid types: `None`, `123` (int), `12.3` (float), `True` (bool)

- Special characters: `"!@#$%"`

- Mixed case: `"MiXeD CaSe"`

Loop:

Iterates through each test case and prints the original value along with uppercase and lowercase conversions. Non-string inputs will show `None` for both conversions.

Task 3 – Test-Driven Development for List Sum Calculator

- Use AI to generate test cases for a function sum_list(numbers) that calculates the sum of list elements.

Requirements:

- Handle empty lists
- Handle negative numbers
- Ignore or safely handle non-numeric values

Example Test Scenarios:

sum_list([1, 2, 3]) → 6

sum_list([]) → 0

sum_list([-1, 5, -4]) → 0

sum_list([2, "a", 3]) → 5

Expected Output 3

- A robust list-sum function validated using AI-generated test cases.

Prompt:

'''

Write a Python program using Test-Driven Development (TDD) to implement a function sum_list(numbers)

that returns the sum of all numeric elements in a list. The function must return 0 for an empty list,

correctly handle negative numbers, and ignore or safely skip non-numeric values such as strings or None.

Generate comprehensive AI-created test cases covering normal cases, edge cases, and mixed-type lists.

Implement the tests using unittest, ensuring all tests validate the correctness of the function.

Provide the complete Python code including the function implementation and the test cases

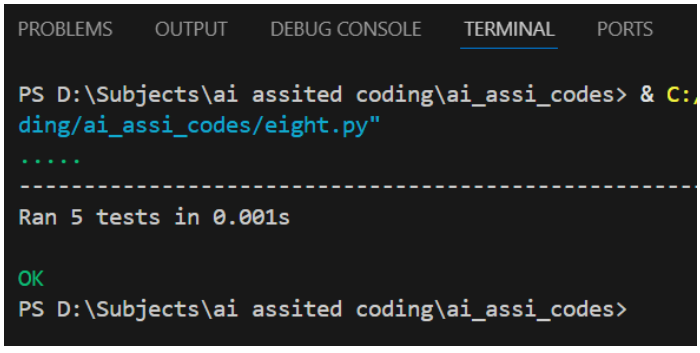
'''

Code:

```
import unittest
def sum_list(numbers):
    if not isinstance(numbers, list):
        return 0
    total = 0
    for num in numbers:
        if isinstance(num, (int, float)):
            total += num
    return total
class TestSumList(unittest.TestCase):
    def test_empty_list(self):
        self.assertEqual(sum_list([]), 0)

    def test_all_numeric(self):
        self.assertEqual(sum_list([1, 2, 3, 4]), 10)
```

```
    def test_mixed_types(self):
        self.assertEqual(sum_list([1, 'a', None, 2.5, '3', -1]), 2.5)

    def test_negative_numbers(self):
        self.assertEqual(sum_list([-1, -2, -3]), -6)

    def test_non_list_input(self):
        self.assertEqual(sum_list("not a list"), 0)
        self.assertEqual(sum_list(123), 0)
        self.assertEqual(sum_list(None), 0)
if __name__ == '__main__':
    unittest.main()
```

Output:



```
'''
Write a Python program using Test-Driven Development (TDD) to implement a function sum_list(numbers)
that returns the sum of all numeric elements in a list. The function must return 0 for an empty list,
correctly handle negative numbers, and ignore or safely skip non-numeric values such as strings or None.
Generate comprehensive AI-created test cases covering normal cases, edge cases, and mixed-type lists.
Implement the tests using unittest, ensuring all tests validate the correctness of the function.
Provide the complete Python code including the function implementation and the test cases
'''

import unittest
def sum_list(numbers):
    if not isinstance(numbers, list):
        return 0
    total = 0
    for num in numbers:
        if isinstance(num, (int, float)):
            total += num
    return total
class TestSumList(unittest.TestCase):
    def test_empty_list(self):
        self.assertEqual(sum_list([]), 0)

    def test_all_numeric(self):
        self.assertEqual(sum_list([1, 2, 3, 4]), 10)

    def test_mixed_types(self):
        self.assertEqual(sum_list([1, 'a', None, 2.5, '3', -1]), 2.5)

    def test_negative_numbers(self):
        self.assertEqual(sum_list([-1, -2, -3]), -6)

    def test_non_list_input(self):
        self.assertEqual(sum_list("not a list"), 0)
        self.assertEqual(sum_list(123), 0)
        self.assertEqual(sum_list(None), 0)
if __name__ == '__main__':
    unittest.main()
```

Code Explanation:

Code implements a sum_list function in Python, along with a set of unit tests using the unittest framework. This follows Test-Driven Development (TDD) principles, where tests define the expected behavior, and the function is implemented to pass those tests. The function is designed to sum only numeric values in a list, handling various edge cases robustly. Below, I'll explain each component step by step.

1. Import Statement

-
-

-
-
- Imports Python's built-in unittest module for writing and running automated tests. This allows validation of the function's correctness.

2. The sum_list(numbers) Function

This function takes a single argument numbers (expected to be a list) and returns the sum of all numeric elements in it. It includes defensive checks to handle invalid inputs gracefully.

- **Input Validation**:
  - if not isinstance(numbers, list): return 0
    - Checks if numbers is a list. If not (e.g., a string, number, or None), it returns 0 immediately. This prevents errors from non-list inputs.
- **Summing Logic**:
  - Initializes total = 0.
  - Loops through each item in numbers using for num in numbers:.
  - For each item, checks if isinstance(num, (int, float)): to ensure it's a number (integer or float).
  - If numeric, adds it to total with total += num.
  - Ignores non-numeric items (e.g., strings, None, booleans) without raising errors.
  - Returns total at the end.
- **Purpose and Behavior**:

    - Safely sums only valid numbers, skipping non-numeric elements.
    - Returns 0 for empty lists or invalid inputs (e.g., not a list).
    - Handles negatives, floats, and mixed types without crashing.
    - Example: sum_list([1, 'a', 2.5, -1]) returns 2.5 (1 + 2.5 + (-1) = 2.5).

3. The TestSumList Test Class

This inherits from unittest.TestCase and defines test methods to verify sum_list works correctly. Each test uses assertions to check expected results.

- **test_empty_list(self)**:
    - Calls sum_list([]) and asserts it equals 0.
    - Purpose: Verifies the edge case of an empty list.
- **test_all_numeric(self)**:
    - Calls sum_list([1, 2, 3, 4]) and asserts it equals 10.
    - Purpose: Tests normal operation with all integers.
- **test_mixed_types(self)**:
    - Calls sum_list([1, 'a', None, 2.5, '3', -1]) and asserts it equals 2.5.
    - Purpose: Ensures non-numeric items are ignored, and mixed types (int, float, negative) are handled.
- **test_negative_numbers(self)**:
    - Calls sum_list([-1, -2, -3]) and asserts it equals -6.

    - Purpose: Confirms negative numbers are summed correctly.
- **test_non_list_input(self)**:
    - Tests invalid inputs: string, integer, and None, asserting each returns 0.
    - Purpose: Validates input type checking.

4. Running the Tests

-
-
-
-
- Runs the tests when the script is executed directly (e.g., python eight.py).
- Discovers and executes all test_* methods, showing pass/fail results.

Overall Behavior and Edge Cases Covered

- **Normal Cases**: Sums integers/floats correctly.
- **Edge Cases**: Empty list (0), non-list inputs (0), mixed types (ignores non-numbers), negatives (included in sum).
- **Why TDD?**: Tests ensure the function is robust and matches requirements (e.g., ignore non-numbers, handle negatives).
- **Potential Improvements**: Could raise errors for non-list inputs instead of returning 0, or handle complex

numbers. Tests could add more boundaries (e.g., very large numbers).

Task 4 – Test Cases for Student Result Class

- Generate test cases for a StudentResult class with the following methods:
- add_marks(mark)
- calculate_average()
- get_result()

Requirements:

- Marks must be between 0 and 100
- Average ≥ 40 → **Pass**, otherwise **Fail**

Example Test Scenarios:

Marks: [60, 70, 80] → Average: 70 → Result: Pass

Marks: [30, 35, 40] → Average: 35 → Result: Fail

Marks: [-10] → Error

Expected Output -4

- A fully functional StudentResult class that passes all AI-generated test

Prompt:

'''

Write a Python program that implements a StudentResult class with the methods add_marks(mark),

calculate_average(), and get_result(). The add_marks method must only accept marks between 0 and 100,

otherwise raise an error. The calculate_average method should compute the average of all stored marks,

and get_result should return "Pass" if average ≥ 40, otherwise "Fail". Generate comprehensive AI-created test cases

covering normal inputs, failing averages, boundary values, and invalid marks. Implement the tests using unittest,

and provide the complete code including the class implementation and all test cases.

'''

Code:

```
import unittest

class StudentResult:
    def __init__(self):
        self.marks = []


    def add_marks(self, mark):
        if not isinstance(mark, (int, float)):
            raise ValueError("Mark must be a number.")
        if mark < 0 or mark > 100:
            raise ValueError("Mark must be between 0 and 100.")
```

```
        self.marks.append(mark)

    def calculate_average(self):
        if not self.marks:
            return 0
        return sum(self.marks) / len(self.marks)

    def get_result(self):
        average = self.calculate_average()
        return "Pass" if average >= 40 else "Fail"
class TestStudentResult(unittest.TestCase):
    def setUp(self):
        self.student = StudentResult()

    def test_add_valid_marks(self):
        self.student.add_marks(85)
        self.student.add_marks(90)
        self.assertEqual(self.student.marks, [85, 90])

    def test_add_invalid_marks(self):
        with self.assertRaises(ValueError):
            self.student.add_marks(-5)
        with self.assertRaises(ValueError):
```

```
            self.student.add_marks(105)
        with self.assertRaises(ValueError):
            self.student.add_marks("not a number")

    def test_calculate_average(self):
        self.student.add_marks(80)
        self.student.add_marks(90)
        self.assertEqual(self.student.calculate_average(), 85)

    def test_calculate_average_empty(self):
        self.assertEqual(self.student.calculate_average(), 0)

    def test_get_result_pass(self):
        self.student.add_marks(50)
        self.student.add_marks(60)
        self.assertEqual(self.student.get_result(), "Pass")

    def test_get_result_fail(self):
        self.student.add_marks(30)
        self.student.add_marks(35)
        self.assertEqual(self.student.get_result(), "Fail")
if __name__ == '__main__':
    unittest.main()
```

Click to add a breakpoint

```
'''
Write a Python program that implements a StudentResult class with the methods add_marks(mark),
calculate_average(), and get_result(). The add_marks method must only accept marks between 0 and 100,
otherwise raise an error. The calculate_average method should compute the average of all stored marks,
and get_result should return "Pass" if average ≥ 40, otherwise "Fail". Generate comprehensive AI-created test cases
covering normal inputs, failing averages, boundary values, and invalid marks. Implement the tests using unittest,
and provide the complete code including the class implementation and all test cases.
'''

import unittest
class StudentResult:
    def __init__(self):
        self.marks = []

    def add_marks(self, mark):
        if not isinstance(mark, (int, float)):
            raise ValueError("Mark must be a number.")
        if mark < 0 or mark > 100:
            raise ValueError("Mark must be between 0 and 100.")
        self.marks.append(mark)

    def calculate_average(self):
        if not self.marks:
            return 0
        return sum(self.marks) / len(self.marks)

    def get_result(self):
        average = self.calculate_average()
        return "Pass" if average >= 40 else "Fail"
class TestStudentResult(unittest.TestCase):
    def setUp(self):
        self.student = StudentResult()

    def test_add_valid_marks(self):
        self.student.add_marks(85)
        self.student.add_marks(90)
        self.assertEqual(self.student.marks, [85, 90])

    def test_add_invalid_marks(self):
        with self.assertRaises(ValueError):
            self.student.add_marks(-5)
```

```
            self.student.add_marks(-5)
        with self.assertRaises(ValueError):
            self.student.add_marks(105)
        with self.assertRaises(ValueError):
            self.student.add_marks("not a number")

    def test_calculate_average(self):
        self.student.add_marks(80)
        self.student.add_marks(90)
        self.assertEqual(self.student.calculate_average(), 85)

    def test_calculate_average_empty(self):
        self.assertEqual(self.student.calculate_average(), 0)

    def test_get_result_pass(self):
        self.student.add_marks(50)
        self.student.add_marks(60)
        self.assertEqual(self.student.get_result(), "Pass")

    def test_get_result_fail(self):
        self.student.add_marks(30)
        self.student.add_marks(35)
        self.assertEqual(self.student.get_result(), "Fail")
if __name__ == '__main__':
    unittest.main()
```

Output:

```
PS D:\Subjects\ai assited coding\ai_assi_codes> & C:/Use
ding/ai_assi_codes/eight.py"
......
----------------------------------------------------------------------
Ran 6 tests in 0.001s

OK
PS D:\Subjects\ai assited coding\ai_assi_codes>
```

Code Explanation:

Code implements a simple StudentResult class in Python, along with a comprehensive set of unit tests using the unittest framework. This follows Test-Driven Development (TDD) principles, where tests are written to validate the functionality before (or alongside) the implementation. Below, I'll break it down step by step, explaining the purpose, structure, and behavior of each component.

1. Import Statement

- This imports Python's built-in unittest module, which provides tools for writing and running automated tests. It's essential for verifying that the code works as expected.

2. The StudentResult Class

This is the core class that manages a student's marks and computes results. It encapsulates data (marks) and provides methods to interact with it safely.

- **__init__(self)** (Constructor):
    - Initializes an empty list self.marks to store the student's marks.
    - Purpose: Sets up the object with no initial data, allowing marks to be added later.
- **add_marks(self, mark)**:
    - Takes a single mark (expected to be a number) and adds it to the self.marks list.
    - Validation:

        - Checks if mark is a number (int or float) using isinstance. If not (e.g., a string), raises a ValueError with the message "Mark must be a number."
        - Checks if mark is between 0 and 100 (inclusive). If outside this range (e.g., -5 or 105), raises a ValueError with "Mark must be between 0 and 100."
    - If valid, appends the mark to the list.
    - Purpose: Ensures only valid marks are stored, preventing invalid data from corrupting calculations. This is a defensive programming technique to handle edge cases early.
- **calculate_average(self)**:
    - Computes the average of all marks in self.marks.
    - If the list is empty, returns 0 (avoids division by zero).
    - Otherwise, uses sum(self.marks) / len(self.marks) to calculate the mean.
    - Purpose: Provides a simple way to get the average mark. Returning 0 for an empty list is a sensible default (e.g., no marks means no average).
- **get_result(self)**:
    - Calls calculate_average() to get the average.
    - Returns "Pass" if the average is >= 40, otherwise "Fail".

    - Purpose: Determines the student's overall result based on a passing threshold (40). This is a business rule that could be adjusted if needed.

3. The TestStudentResult Test Class

This inherits from unittest.TestCase and defines test methods to verify the StudentResult class behaves correctly. Each test method starts with test_ and uses assertions to check expected vs. actual results. The setUp method runs before each test to create a fresh StudentResult instance.

- **setUp(self)**:
    - Creates a new self.student instance for each test.
    - Purpose: Ensures tests are isolated (no shared state between tests).
- **test_add_valid_marks(self)**:
    - Adds two valid marks (85 and 90) and asserts that self.student.marks equals [85, 90].
    - Purpose: Verifies that valid marks are added correctly.
- **test_add_invalid_marks(self)**:
    - Uses self.assertRaises(ValueError) to check that invalid inputs (negative number, >100, or non-number) raise the expected error.
    - Purpose: Ensures error handling works for boundary and invalid cases.
- **test_calculate_average(self)**:

    - Adds marks (80 and 90), calculates the average, and asserts it equals 85.
    - Purpose: Verifies average calculation for normal cases.
- **test_calculate_average_empty(self)**:
    - On an empty student, asserts the average is 0.
    - Purpose: Tests the edge case of no marks.
- **test_get_result_pass(self)**:
    - Adds marks averaging to >=40 (50 and 60), asserts result is "Pass".
    - Purpose: Verifies passing logic.
- **test_get_result_fail(self)**:
    - Adds marks averaging to <40 (30 and 35), asserts result is "Fail".
    - Purpose: Verifies failing logic.

Task 5 – Test-Driven Development for Username Validator

Requirements:

- Minimum length: 5 characters
- No spaces allowed
- Only alphanumeric characters

Example Test Scenarios:

is_valid_username("user01") → True

is_valid_username("ai") → False

is_valid_username("user name") → False

is_valid_username("user@123") → False

Expected Output 5

A username validation function that passes all AI-generated test cases.

Prompt:

'''

Write a Python program using Test-Driven Development (TDD) to implement a function is_valid_username(username)

that validates usernames based on the following rules: minimum length of 5 characters,no spaces allowed, and only alphanumeric

characters are permitted. Generate comprehensive AI-created test cases that include valid usernames, too-short usernames,

usernames with spaces, and usernames containing special characters. Implement the tests using pytest to ensure the function

passes all scenarios. Provide the complete Python code including the validation function and the corresponding test cases.

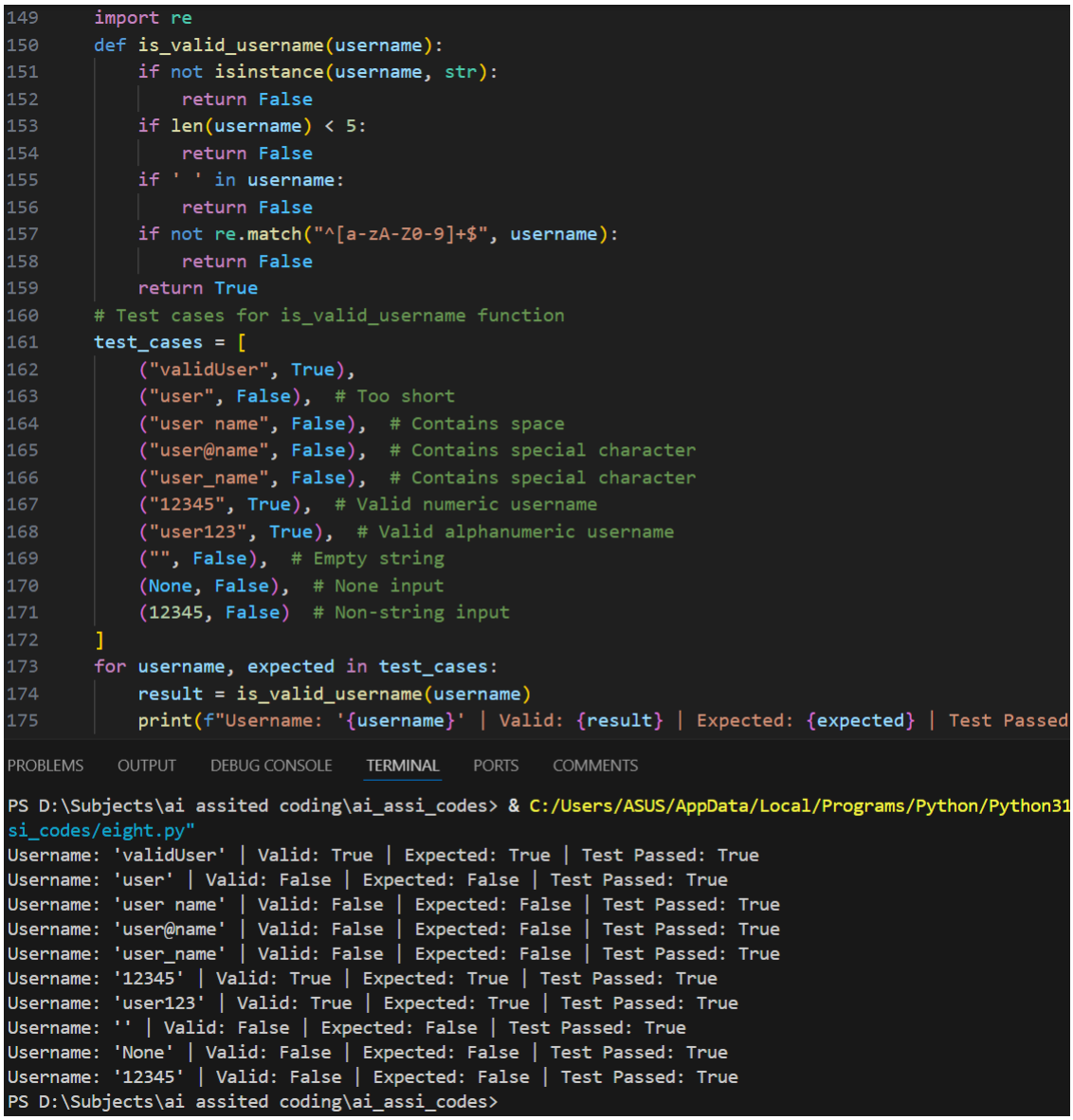
'''

Code:

```
import re
def is_valid_username(username):
    if not isinstance(username, str):
        return False
    if len(username) < 5:
        return False
    if ' ' in username:
        return False
    if not re.match("^[a-zA-Z0-9]+$", username):
        return False
    return True
# Test cases for is_valid_username function
test_cases = [
    ("validUser", True),
    ("user", False),  # Too short
    ("user name", False),  # Contains space
    ("user@name", False),  # Contains special character
    ("user_name", False),  # Contains special character
    ("12345", True),  # Valid numeric username
    ("user123", True),  # Valid alphanumeric username
    ("", False),  # Empty string
    (None, False),  # None input
```

```
    (12345, False)  # Non-string input
]
for username, expected in test_cases:
    result = is_valid_username(username)
    print(f"Username: '{username}' | Valid: {result} | Expected: {expected} | Test Passed: {result == expected}")
```

Output:



Code Explanation:

The provided code defines a function `is_valid_username` that checks if a given username meets specific criteria: it must be a string, at least 5 characters long, contain no spaces, and consist solely of alphanumeric characters. The function uses regular expressions to validate the alphanumeric condition. A series of test cases are created to evaluate the function against various scenarios, including valid usernames, too-short usernames, usernames with spaces, and those containing special characters. Each test case is executed in a loop, where the function's output is compared against the expected result, and the outcome of each test is printed to the console, indicating whether the test passed or failed.